

Private Information Retrieval based on Homomorphic Encryption, Revisited

Jaeseon Kim¹ , Jeongeun Park² , and Hyewon Sung¹ 

¹ Ewha Womans University, Seoul, Republic of Korea

`jaeseon.kim@ewha.ac.kr, hyewonsung@ewha.ac.kr`

² Norwegian University of Science and Technology (NTNU), Trondheim, Norway

`jeongeun.park@ntnu.no`

Abstract. Private information retrieval (PIR) enables a client to retrieve data from a server while preserving the confidentiality of the client’s query. When PIR is instantiated with fully homomorphic encryption (FHE), the protocol becomes non-interactive, requiring only a query-answer exchange, and it achieves asymptotically optimal communication and computation complexity. Although several FHE-based PIR protocols have been practically implemented with the desired properties, there has been little detailed comparison among them. As a result, it remains unclear which protocol is most efficient in practice with respect to various aspects such as performance and scalability.

In this paper, we revisit existing protocols by categorizing them into two different structures in order to analyze the advantages and disadvantages of each class in detail, with a focus on practical implementations. Furthermore, we introduce and compare various homomorphic algorithms that can be utilized for query optimization, discussing the strengths and limitations of each. Finally, with the goal of identifying the most efficient protocol in terms of computational cost and memory usage, based on database size. Additionally, we address common misconceptions that may lead to inefficient choices in real-world deployment scenarios and offer the best solutions. Consequently, our analysis and experimental results demonstrate that the less-explored design achieves a 90% reduction in communication cost and an 8× decrease in computational overhead compared to the other one, challenging the common misconception.

Keywords: Private Information Retrieval, PIR, Homomorphic Encryption, FHE

1 Introduction

Retrieving information from a remote server might leave some identities or interest of clients to a database(DB) holder, which would cause serious privacy issues in many cases. For example, medical data can leak sensitive health information, and accessing patterns of certain database may reveal a private interest. In order to avoid such privacy problems, private information retrieval (PIR) is often considered, which allows clients to obtain desired items from a remote database owned by a server, without revealing any information about the items to the server. PIR itself allows a direct application; private database queries, moreover, being an important building block of many privacy-preserving protocols such as contact discovery [7], anonymous messaging [33,28,5], safe browsing [27] and the list goes on.

After Chor et al. [17] introduced the first concept of PIR, it has been studied continuously, improving communication complexity or computation complexity or both, assuming either single server or multiple servers. The single server scenario is simpler to build and more practically feasible due to its simple security model. Since only cryptographic approaches allow efficient single server scenarios as proved in [17], the efficiency of such protocols have been accelerated over the last decades [3,30,23,20,36,34,32,39,21,25,29,31,10].

Especially PIR based on homomorphic encryption which allows computation over encrypted data, offers optimal communication/computation complexity and non-interactive protocol³. Keeping clients offline as

³ There is no additional interaction between a client and the data holder other than one round query-response.

long as they want is indeed a practically attractive advantage for real-world cloud service providers such as Google, Apple etc.

Recently, Apple announced a new feature [1] employing PIR based on homomorphic encryption to enhance privacy for clients to receive email without revealing any information about the sender to Apple server. As FHE-based PIR is being ready to be deployed and its importance has been emphasized rapidly, it is time to be more precise on what has been done in the literature and provide the best solutions for practical deployment.

The recent research trend on FHE-based PIR has been moving towards stateful designs [37,34,25,39,21,10,29] from existing stateless protocols where client may store information between queries remembering previous interactions, in order to achieve sublinear computational complexity⁴. It is a natural research stream since stateless protocol is the main building block of stateful designs, and the current practical improvements of stateless protocols can accelerate the running time of stateful protocols combined with its sublinear complexity.

However, a noticeable fact here is that almost all practical implementations of such designs have the same base structure which we call hypercube design in this paper, where the database structure is particularly structured as a d -dimensional hypercube, where $d \geq 2$. In fact, it is the base structure of XPIR [3]/SealPIR [5] and starting from $d = 2$, further improved by OnionPIR [34] which achieved $\log n$ communication complexity by setting $d = O(\log n)$ in the same structure, where n is the size of database. It is a plausible result that optimal communication complexity has been achieved, so that further optimizations such as building stateful designs are the remaining part for real-world deployment.

However, we would like to highlight a previously proposed stateless design, SHECS-PIR [36], which had already achieved optimal communication complexity without relying on any specific data structure [34]. In this work, we classify SHECS-PIR as a tree-based PIR due to its use of tree-based computation. To be more precise, SHECS-PIR works as a hardware look-up table reading in a privacy-preserving manner. Data elements are structured as a look-up table, and the desired item is obviously selected through $\log n$ rounds of binary gates where data items are compared in pairs in every round with each corresponding encrypted bit given by a client, and the number of selected items is halved after each round.

Therefore, the constant hidden in the asymptotic complexity $O(\log n)$ is exactly 1, whereas the hypercube-based approach still adds a large constant depending on the implementation parameters. Despite its clear advantage in terms of communication efficiency⁵, this approach has received little attention in subsequent research.

As briefly mentioned earlier, the practical deployment of FHE-based PIR is becoming increasingly feasible, as evidenced by recent developments such as Apple’s new feature. To provide concrete and precise solutions for real-world applications, it is crucial to thoroughly investigate existing designs, conduct rigorous analyses, and offer concrete parameters depending on database sizes, to avoid inefficiency caused by wrong selections of algorithms and parameters.

Therefore, we aim to revisit the two designs and provide a careful consideration of the trade-offs associated with various optimization techniques, all within the context of practical deployment.

1.1 Our contribution

In this paper, we revisit a previously underexplored design and provide a comprehensive analysis of its distinctions compared to hypercube-based structures. Based on this analysis, we introduce a new perspective on FHE-based PIR, highlighting the best methods depending on the size of the database and the type of database (public/private).

In general, PIR assumes access to a public database. However, accessing private databases for outsourced computation has gained significant attention in recent years. A typical use case involves clients downloading their encrypted data from a cloud and sending it to a computing party for tasks such as machine learning inference or other applications.

⁴ In fact, in the stateless design, by default, the main computation has to go thorough all database, resulting in $O(n)$ complexity, which is optimal.

⁵ transmitting exactly $\log n$ encrypted bits and matching the communication cost of non-private PIR if further optimization such as transciphering[35] is used.

Therefore, in Section 7, we consider both scenarios in our discussion, as the optimization techniques and performance characteristics vary depending on the underlying PIR structure. In Section 8, we present the optimal parameter sets for the best query unpacking algorithm for each (reasonably large) database size.

To begin, we examine the key differences between the two fundamental PIR designs: hypercube-based PIR and tree-based PIR. Notably, the two concurrent FHE-based PIR protocols, SHECS-PIR[36] and OnionPIR[34], exhibit the same asymptotically optimal communication complexity. However, based on the running time of their implementations, OnionPIR appears to outperform SHECS-PIR, as briefly mentioned in [34] without any further discussion.

However, we have identified two factors that may underestimate the potential benefits of tree-based approach (SHECS-PIR), which can actually surpass hypercube-based one (OnionPIR) in terms of both computation and communication costs. The key observations leading to our research are as follows:

1. The number of ciphertexts required by OnionPIR for a single query is $128 + 4 \log_4(n/128)$, whereas SHECS-PIR requires exactly $\log_2 n$ ciphertexts, although both have logarithmic communication complexity, i.e., $O(\log n)$.
2. OnionPIR employs a faster optimization algorithm for the query compared to SHECS-PIR, which results in a slower running time for SHECS-PIR.

Now, it has become evident that the actual communication cost of both designs is different significantly, which also results in increased computation time when the query information is packed into a single ciphertext. In this scenario, the server must invest additional computational resources to unpack the ciphertext into $O(\log n)$ individual ciphertexts before proceeding with the main computation. This process is referred to as query unpacking. Consequently, the computation time increases linearly with the number of ciphertexts required for a single query. The following our key findings are worth noting:

1. Several algorithms, specifically designed for lattice-based FHE schemes, can be utilized for query unpacking in the optimization of PIR queries. However, only one has been using continuously in the literature due to its optimal computation complexity.
2. Query unpacking and the primary PIR protocols are independent of one another, meaning any unpacking algorithm can be applied to any protocol design.
3. The optimal unpacking algorithm, which maximizes the overall computational efficiency of the protocol, differs depending on not the PIR design itself but the size of the database.

Since both designs have the same multiplication depth $O(\log n)$, the noise of the main PIR protocol grows in the same scale. However, depending on query unpacking algorithms, the noise contained in the unpacked query differs, resulting in different performance due to the different capacity of noise level that the main computation can handle. Considering the impact of the pre-computation phase, our main contribution based on the above findings is as follows:

- We categorize existing PIR protocols based on FHE schemes into two different structures (Sections 3 and 4).
- We introduce three main algorithms for query unpacking, and provide pros and cons of each (Section 5).
- We provide a comprehensive analysis for both PIR fundamental designs in theory (Section 7), and in a practical scenario (Section 8). As a result, tree-based structure outperforms hypercube based structure in both communication and computation in the case of private database. Our experimental result with concrete parameters show that tree-based one achieves 90% lower communication cost, and 8x lower computational overhead. In the public database setting, hypercube-based PIR offers more flexibility in parameter selection and demonstrates better performance than the tree-based structure when parameters are carefully chosen.
- We suggest the best query unpacking algorithm depending on the database size in terms of the whole computation time with concrete parameter sets. It turns out that the most commonly used algorithm is not the best one in PIR context due to its high noise growth. As a result, we show the unexplored algorithm performs better in relatively large database in Section 8.

1.2 Threat Model

In FHE-based PIR protocols, server who owns database is considered as semi-honest adversary who wants to learn about clients' queries however, do not deviate the protocol. In other words, the adversary follows the protocol honestly, but is allowed to access to and investigate public information such as ciphertexts, public keys. Therefore, IND-CPA secured FHE schemes can guarantee the security against such adversary by default.

All researches that we are referring to and employing here have already proved their security under the security of their underlying FHE. Therefore, we do not discuss the security again in this paper. We note that using FHE solely cannot guarantee fully malicious security where an adversary can modify ciphertexts on its own since FHE allows malleability by its definition. Therefore, it requires other cryptographic primitives such as zero knowledge proof, which is not our focus.

1.3 Paper Organization

In Section 2, we study basic algorithms of homomorphic encryption, which are building blocks of FHE-based PIR protocols that we present. We introduce the hypercube based PIR protocols such as SealPIR and OnionPIR and how the optimization for communication is achieved in this structure in Section 3. Then, we introduce two similar tree-based PIR protocols such as SHECS-PIR and the reading part of FHE-based ORAM called panacea in Section 4. We provide three different algorithms which can be used for efficient query unpacking in Section 5 with detailed comparison among them. We provide a note about neglected but very important parameters for FHE based PIR in Section 6. The detailed analysis and comparison among different structures and tree-based PIR is discussed in Section 7. We also offer experimental result with reasonably large database size and element size matches the real-world application [1], suggesting the best parameter sets per database size in Section 8. And we conclude the paper with a further discussion for both designs in Section 9.

2 Preliminaries

2.1 Notation and Lattice Problem

Notation: We denote λ as the security parameter. We define vectors and matrices in lowercase bold and uppercase bold, respectively. Dot product of two vectors $\mathbf{v}, \mathbf{w}$ is denoted by $\langle \mathbf{v}, \mathbf{w} \rangle$. For a vector $\mathbf{x}$, both $\mathbf{x}[i]$ and $\mathbf{x}_i$ denotes the i -th component scalar. Similarly, for a matrix $\mathbf{X}$, $\mathbf{X}[i]$ denotes the i -th row vector. We denote $\mathcal{R}$ as a polynomial ring defined as $\mathbb{Z}/(X^N + 1)$ and $\mathcal{R}_q$ as $\mathcal{R} \bmod q$, where N is a power of 2. For a real $\alpha > 0$, D_α denotes the Gaussian distribution of standard deviation α over $\mathcal{R}$. Let $\mathbf{g} := [1, g, g^2, \dots, g^{\ell-1}]$ be called a gadget vector, for a positive integer g and $\ell = O(\log_g q)$, and define $\mathbf{G} := \mathbf{I} \otimes \mathbf{g}^t$ as a gadget matrix. $\text{Var}(\mathbf{X})$ denotes the variance of a random variable $\mathbf{X}$, and $\text{Err}(\mathbf{c})$ denotes the noise contained in a FHE ciphertext $\mathbf{c}$.

2.2 Homomorphic Encryption

Homomorphic encryption is a strong cryptographic tool which allows a computation over encrypted data. We introduce algorithms of homomorphic encryption based on RLWE assumption, which are practically optimized and commonly used in privacy preserving protocols including PIR protocols, in this section. We describe probabilistic polynomial time (PPT) algorithms of RLWE based FHE schemes, as well as several types of ciphertexts, each of which is used for different purpose in a PIR protocol.

- **FHE.Setup**(1^λ) $\rightarrow$ **params** : It takes the security parameter λ on input and outputs the system (public) parameters **params** including the ciphertext modulus q , plaintext modulus p , a gadget vector $\mathbf{g}$, and the key distribution D_α . We assume that all the algorithms defined below implicitly takes **params** as inputs.
- **FHE.KeyGen**(**params**) $\rightarrow$ (**sk**, **evk**) : it generates $\text{sk} := s \leftarrow D_\alpha$, and public evaluation key **evk**.

- $\text{FHE.RLWE.Enc}(\text{sk}, \Delta \cdot m) \rightarrow \text{ct} := (a, b)$: it generates an RLWE ciphertext $\text{ct} := (a, b)$ encrypting a message $m \in \mathcal{R}_p$ by computing $b = a \cdot s + \Delta m + e$, where $a \leftarrow \mathcal{R}_q$ at uniformly random, $e \leftarrow D_\alpha$ and a positive integer $\Delta = q/p$ is called a scaling factor.
- $\text{FHE.RGSW.Enc}(\text{sk}, m) \rightarrow \mathbf{C} := \mathbf{Z} + m \cdot \mathbf{G} \in \mathcal{R}_q^{2\ell \times 2}$: it computes an RGSW ciphertext which is a vector of dimension of 2ℓ consisting of RLWE ciphertexts encrypting 0 (denoted as $\mathbf{Z}$) adding a message $m \in \mathcal{R}$ scaled with the gadget matrix $\mathbf{G}$.
- $\text{FHE.ksk.Gen}(\text{sk}, \text{info}(\text{sk})) \rightarrow \text{ksk} := (\mathbf{z} + \mathbf{g} \cdot \text{info}(\text{sk}))$: it outputs a key switching key ksk encrypting partial (possibly whole) information of sk (denoted as $\text{info}(\text{sk})$). The key switching key is a form of a vector of dimension ℓ , consisting of RLWE ciphertexts encrypting 0 (denoted as $\mathbf{z}$) adding key information scaled with the gadget vector $\mathbf{g}$. Note that the size of the key switching key is half of an RGSW ciphertext.
- $\text{FHE.Add}(c_1, c_2) \rightarrow c_+ \in \mathcal{R}_q^2$: it takes two RLWE ciphertexts $c_1 := (a_1, b_1)$ and $c_2 := (a_2, b_2)$ encrypting m_1 and m_2 respectively, it outputs an RLWE ciphertext c_+ encrypting $m_1 + m_2$, by computing $c_t := (a_1 + a_2, b_1 + b_2)$.
- $\text{FHE.PlainMult}(c, T(X)) \rightarrow c' \in \mathcal{R}_q^2$: it takes an RLWE ciphertext $c := (a, b)$ encrypting m and a polynomial $T(X) \in \mathcal{R}_q$, and the algorithm outputs an RLWE ciphertext encrypting $m \cdot T(X)$, by computing $c' := (a \cdot T(X), b \cdot T(X))$. This algorithm is called ciphertext-plaintext multiplication.
- $\text{FHE.ExternalProd}(c, \mathbf{C}) \rightarrow c' \in \mathcal{R}_q^2$: it takes an RLWE ciphertext c and an RGSW ciphertext $\mathbf{C}$ encrypting m_1 and m_2 respectively, and outputs an RLWE ciphertext c' encrypting $m_1 \cdot m_2$, by computing $c' := \mathbf{G}^{-1}(c) \cdot \mathbf{C}$, where $\mathbf{G}^{-1}$ is an inverse of the gadget matrix $\mathbf{G}$.
- $\text{FHE.Dec}(c, \text{sk}) \rightarrow m$: it takes an RLWE ciphertext $c := (a, b)$ and secret key $\text{sk} := s$, and outputs a message $m \in \mathcal{R}_p$ by computing $\lfloor \frac{1}{\Delta}(b - a \cdot s) \rfloor$.
- $\text{FHE.SampleExtract}(c, i) \rightarrow \mathbf{c}$: it takes an RLWE ciphertext encrypting a polynomial $M(X)$, $c = \text{FHE.RLWE.Enc}(\Delta \cdot M(X)) := (\sum_{i=0}^{N-1} a_i X^i, \sum_{i=0}^{N-1} a_i X^i \cdot \sum_{i=0}^{N-1} s_i X^i + \Delta \cdot M(X) + e(X))$, and an index $0 \leq i < N$, and outputs $\mathbf{c} = (a'_0, \dots, a'_{N-1}, b)$, where $b = \sum_{j=0}^{N-1} a'_j \cdot s_j (= \sum_{j=0}^i a_{i-j} \cdot s_j - \sum_{j=i+1}^{N-1} a_{N+i-j} \cdot s_j) + \Delta \cdot M_i + e$, and M_i is the i -th coefficient of $M(X)$, which is called a LWE ciphertext. It extracts one coefficient of a message polynomial encrypted as an RLWE ciphertext into an LWE ciphertext.
- $\text{FHE.Mux}(\mathbf{C}, \mathbf{a}, \mathbf{b}) \rightarrow \mathbf{C} \cdot (\mathbf{b} - \mathbf{a}) + \mathbf{a}$, where $\mathbf{C}$ is an encryption of a bit, and $\mathbf{a}$ encrypting m_a and $\mathbf{b}$ encrypting m_b , and it outputs an encryption of m_a if $\mathbf{C}$ is an encryption of 0, m_b , otherwise. It takes one homomorphic multiplication and two homomorphic additions.

3 Hypercube based PIR

PIR protocols in this category has the database structure which is a d -dimensional hypercube suggested by Stern [38] to originally achieve the sublinear communication complexity. It has been improved from $O(\sqrt{n})$ [2,38,3,5] to $O(\log n)$ [34], where n is the number of data elements which a server holds.

3.1 PIR Protocol Description

- **Setup:** Server determines the number of dimensions of the database hypercube d , depending on the size of database and FHE parameters that a client wants to use and publishes it. Then the server sets the database structure into the amenable format a $(D_1 \times D_2 \times \dots \times D_d)$ -hypercube).
- **Query Generation:** Client choose an index i which is represented as the data point in the hypercube structure. The index i is encoded as a tuple of bit strings $(I_1, I_2, \dots, I_d)$, where I_j is a bit string of length $O(\sqrt[n]{n})$, and each bit string can be considered as a unit vector since there is only one 1 to represent which position he wants to touch in the corresponding dimension. The client encrypts all the bit strings under FHE and sends them (denoted by $q := (q_1, \dots, q_d)$) to the server.
- **Server Computation:** The server computes matrix-vector multiplication between the initial dimension (D_1) of the hypercube and the corresponding query vector. Then, the server iteratively repeat the same process with the $(D_2 \times D_3 \times \dots \times D_d)$ -hypercube for $j \in [2, \dots, d]$ until the multiplication is done with the last query q_d . The server returns the output to the client.

- **Response Decoding:** The client decrypts the received answer from the server, encrypted under FHE with its own key.

Initial practical instantiation (XPIR [2] and SealPIR [5]) based on BGV [8] and FV scheme [22], respectively, set d as 2 or 3 to manage the noise growth since the homomorphic multiplication of the underlying FHE scheme incurs large (quadratic) noise growth. Furthermore, for small d , they used ciphertext-plaintext multiplications which give much less noise than ciphertext-ciphertext multiplication. As a trade-off, ciphertext expansion factor, denoted by F , becomes a critical factor for server’s response (in fact, the size of server’s answer is proportional to F^{d-1}).

In more detail, for the case of $d = 2$, after the first dot product with row query vector (ciphertext) and the $\sqrt{n} \times \sqrt{n}$ matrix consisting of data elements in clear-text, the output is a ciphertext vector because of homomorphic encryption. For the next dot product between the output vector and the column query, they divide the output each element into F chunks, resulting in $F \times \sqrt{n}$ matrix encrypting the same underlying messages. Therefore, the final output which server sends to the client is F ciphertexts.

3.2 Optimization for communication complexity

Query: From $O(\sqrt{n})$ to $O(\log n)$ With XPIR’s d -hypercube database, clients prepare $d\sqrt[n]{n}$, especially if $d = 2$, only $2\sqrt{n}$ FHE ciphertexts are required. Later SealPIR introduced a way to represent the index per dimension encoded into a monomial, therefore, only one polynomial per dimension was necessary for sufficiently large size of polynomial degree N . Consequently, the actual size of query is reduced to d , the complexity is further reduced to $O(d\sqrt[n]{n}/N)$.

One can optimize communication complexity which is logarithmic in n by adjusting d (the number of dimension) of the hypercube by setting all dimensions small constant. In other words, $d = O(\log n)$ implies each dimension $D_i = O(\sqrt[\log n]{n}) = O(1)$ (it is easy to see if $n = 2^c$ for some constant c). To reduce the communication cost, and computation cost for generating query on the client’s side, it is better to choose small constants for each dimension. In fact, the recommended parameters for OnionPIR [34] which firstly introduced this method, is $D_1 = 128, D_2 = D_3 = D_4 = \dots = D_d = 4$. Consequently, the number of dimensions $d = 1 + \lceil \log_4(n/128) \rceil$, which is logarithmic in n .

Server’s Reply: From $O(F^{d-1})$ to $O(1)$ OnionPIR also reduced the size of server’s reply incurred from the previous methods like in XPIR and SealPIR, by employing GSW-like homomorphic multiplication [24], denoted as FHE . ExternalProd in Section 2. This algorithm is beneficial when there is a series of multiplication performed sequentially due to its additive noise growth. The method is already given by SHECS-PIR instantiated with TFHE [15] where efficient such product called external product was designed and later another faster design was introduced in [6], therefore SHECS-PIR was the non-interactive PIR protocol which optimized communication complexity for the first time, which will be discussed in Section 4 in detail.

In order to support this multiplication efficiently, there should be specific ciphertext structures, where one operand is an RLWE ciphertext and the other is an RGSW ciphertext. In more detail, the query which encrypt bits should be formed as RGSW ciphertexts to make use of full advantages of the operation. OnionPIR represents information of the index i in bits, hence the query consists of RGSW ciphertexts, resulting in preparing $Q := \sum_{j=1}^d D_j = 128 + 4 \cdot (d - 1)$ RGSW ciphertexts.

However, this approach gives high communication cost since an RGSW ciphertext encrypting only a bit costs few hundreds KB, which is not ideal in terms of throughput. Therefore, OnionPIR allowed the client to pack Q ciphertexts in one ciphertext as SealPIR optimized query size of XPIR, and this method is called query packing. Therefore, the query complexity is reduced to $O(d\sqrt[n]{n}/N)$ from $O(d\sqrt[n]{n})$ in SealPIR, and to $O(\log n/N)$ from $O(\log n)$ in OnionPIR, where N is the degree of plaintext polynomial.

As a result, server has to unpack these query additionally to make them amenable format to proceed the main computation between a query vector and the data stored in the corresponding dimension, which is called query unpacking. Since this step is commonly used in all protocols that we discuss in this paper, this will be discussed in a separate section (Section 5) in detail.

4 Tree-based PIR

PIR protocols in this category do not need a particular database structure. For the computation, the database is prepared as a look-up table (LUT) and each index is a form of bit representation. A client sends the index i which is represented as bits, encrypted, and the server homomorphically selects the i -th item then gives the result to the client. This protocol emulates the non-private version of PIR, therefore, the communication complexity is optimal $O(\log n)$ (i.e. a client only sends $\log n$ ciphertexts). We study the first kind of such PIR, called SHECS-PIR [36] mainly in this paper.

However we note that a similar variant exists, which is used to build a non-interactive and stateless oblivious RAM (ORAM) [18]. In other words, the reading part of their construction is also considered as a tree-based PIR. The main difference is that their design first generates an unit vector via tree structure, then perform a dot product between the unit vector and the database. It can be a potential computationally cheaper approach since the database is touched only once. However, we do not investigate the practical differences between the two similar approach in this paper.

4.1 SHECS-PIR

The protocol description is as follows:

- **Setup:** Server sets the database as a look-up table, each corresponding index is represented as bits.
- **Query Generation:** Client choose an index i which is represented as bits, such as $j = \sum_{j=0}^s \epsilon_j \cdot 2^j$, where $s = \lceil \log n \rceil - 1$, and encrypts ϵ_j individually, with FHE. Then it sends the query $q := (\text{FHE}.\text{Enc}(\epsilon_0)), \dots, \text{FHE}.\text{Enc}(\epsilon_s))$ consisting of $\log n$ ciphertexts to the server, where $\text{FHE}.\text{Enc}$ is an FHE encryption algorithm.
- **Server Computation:**
 - It runs $\text{FHE}.\text{Mux}(\text{FHE}.\text{Enc}(\epsilon_0), d_{2j}, d_{2j+1})$ for $j \in [0, \dots, n/2 - 1]$ to select an encryption of data of which the lest significant bit equals to ϵ_0 , obviously, where d_j 's are data elements.
 - After the first round of homomorphic mux gates with n clear data elements which are not encrypted, the output selected $n/2$ elements are encrypted due to homomorphic operations.
 - Then it runs again the same round with the $\text{FHE}.\text{Enc}(\epsilon_1)$ over $n/2$ elements to select the half whose the second least significant bit equals to ϵ_2 , homomorphically. The process is repeated for $\log n$ times to select the right element (the i -th element), which is an encryption of d_i .
- **Response Decoding:** The client decrypts the received answer from the server, encrypted under FHE with its own key (in detail, running $\text{FHE}.\text{Dec}(c, \text{sk})$, where c is the answer given by the server.

4.2 Read of Panacea

Panacea [18] is designed for oblivious ram (ORAM), which is why all the database elements are already encrypted under client's key. However, if we use this part for PIR, database is not necessarily encrypted, therefore, which gives much more improvement in terms of computation cost, than its original application; ORAM.

The client's part is as same as SHECS-PIR, so we discuss the main protocol of server in this paper in the following:

- **Server Computation:**
 - It runs homomorphic de-multiplexer [18] to generate a unit vector $\mathbf{u}$ where the i -th component is an encryption of 1 and the rest are encryptions of 0.
 - It computes dot product between the unit vector and the array of database elements. (i.e. computing $r := \langle \mathbf{u}, \mathbf{d} \rangle$, where $\mathbf{d}$ is a vector of database elements.)
 - send the r to the client.

The client runs $\text{FHE}.\text{Dec}(r, \text{sk})$ to obtain the desired data element d_i . We note that homomorphic de-multiplexer consists of $n/2$ external products and the dot product is performed between ciphertext (the unit vector) and the data elements (plaintext).

5 Query unpacking

All of the above mentioned PIR protocols employ query packing technique which encodes all the information about the query in one polynomial, resulting in one ciphertext, in order to optimize the actual communication cost as well as the complexity. For example, in SealPIR as we briefly mentioned in Section 3, each query representing the corresponding dimension consists of bit information, therefore, a monomial is enough to contain the information. Therefore, instead of sending $d \sqrt[n]{n}$ ciphertexts, d ciphertexts were sent. In SHECS-PIR, the bit representation of the index, consisting of exactly $\log n$ bits is the query. Instead of sending $\log n$ ciphertexts, it embeds all bits into the coefficients of a polynomial, resulting in one ciphertext for the query.

This process is called ciphertext packing on the client's side. As soon as the server received the packed query, he has to unpack the query to turn it into a necessary format to process the main computation. The additional step is called query unpacking.

5.1 Subalgorithms for query unpacking

In all RLWE based PIR protocols that we present, a packed ciphertext is a form of RLWE ciphertext, and the actual query used for the main computation is a form of RGSW ciphertext to allow external products which provides lower noise growth and faster computation time than RLWE-RLWE multiplication. Therefore, the query unpacking is an algorithm to convert an RLWE ciphertext to several RGSW ciphertexts. In other words, it boils down to build an efficient conversion algorithm from an RLWE to an RGSW ciphertext. In this section, we recall existing efficient such conversion algorithm and analyze the pros and cons of each algorithm per PIR design. Here, we rule out the most straightforward way called circuit bootstrapping introduced in [15] since it is the most expensive such algorithm and we are interested in more efficient ways.

In more detail, we want to start with an RLWE ciphertext encrypting a polynomial $M(X)$ of degree up to $k-1$, and end up with k RGSW ciphertexts encrypting k coefficients of $M(X)$. Then we divide the conversion into two sub-phases: 1) a conversion from an $\text{FHE} \cdot \text{RLWE} \cdot \text{Enc}(M(X))$ to an $\text{FHE} \cdot \text{RLWE} \cdot \text{Enc}(M_i)$, where M_i is the i -th coefficient of $M(X)$ for $i \in \{0, \dots, k-1\}$, and 2) a conversion from ℓ $\text{FHE} \cdot \text{RLWE} \cdot \text{Enc}(g^j \cdot M_i)$, for $j \in [\ell]$, to an $\text{FHE} \cdot \text{RGSW} \cdot \text{Enc}(M_i)$. We note that an RGSW ciphertext consists of $2 \cdot \ell$ rows of a polynomial vector of dimension 2. The upper ℓ rows are not exactly RLWE ciphertexts, since the message is encoded on the first component scaled with different value g^j . We introduce existing efficient algorithms for both sub-phases below.

Conversion from RLWE to RLWE

Private Functional Key switching introduced in TFHE It is the initial such algorithm introduced in an early version of the TFHE paper [14], which was directly used for the unpacking algorithm of SHECS-PIR. The algorithm (Algorithm 1) consists of homomorphic additions therefore, it is expected to be faster than running one circuit bootstrapping. We show the algorithm slightly modified for PIR unpacking specifically. Since the decomposition is not exact but approximate, the parameter t plays a key role to guarantee correctness. In other words, the decomposition grabs the upper t bits of each input, therefore the rest is considered as the noise term, hence it is important to keep it small enough to handle further operations. In practice, t is chosen close to $\log_2 q$ (see experimental result of SHECS-PIR). Then the downside of large chosen t is the computation time since the algorithm runs $t \cdot N$ homomorphic addition over RLWE ciphertexts. Depending on the depth of the main PIR protocol, t can be chosen small enough, then the algorithm can be more beneficial than the others due to its cheap operation.

RLWE($M(X)$) to RLWE(M_i) Algorithm 2 also gives the same functionality which transforms an RLWE ciphertext encrypting a polynomial $M(X)$ into an RLWE ciphertext encrypting i -th coefficient of $M(X)$, where $i \in \{0, \dots, N-1\}$. The algorithm is also widely used in the literature [11, 18, 19]. It mainly consists of scalar product between an integer in $\mathbb{Z}_q$ and a polynomial in $\mathcal{R}_q$. This operation is cheaper than polynomial multiplications, however, more expensive than additions over $\mathcal{R}_q$.

Algorithm 1 PrivKeySwitch: Private Functional Key Switching [14]

Input: $c := \text{FHE.RLWE.Enc}(s, \Delta \cdot M(X)) \in \mathcal{R}_q^2$ and a key switching key $\text{ksk} := \{\text{FHE.RLWE.Enc}(2^i \cdot s_j)\}_{i \in [t], j \in [0, \dots, N-1]}$, where s_j is j -th coefficient of the secret key s , and an index $k \in \{0, \dots, N-1\}$.
Output: $\text{FHE.RLWE.Enc}(s, \Delta \cdot M_k)$

```

1: Run SampleExtract( $c, k$ )  $\rightarrow (a_0, \dots, a_{N-1}, b)$ 
2: for  $i \leftarrow 0, \dots, N-1$  do
3:   Let  $\tilde{a}_i := a_i - (a_i \bmod 2^{\lceil \log q \rceil - t})$   $\triangleright$  grabbing the upper  $t$  bits of  $a_i$ 
4:   Decompose  $\tilde{a}_i = \sum_{j=\lceil \log_2 q \rceil - t + 1}^{\lceil \log_2 q \rceil} a_{i,j} \cdot 2^j$   $\triangleright a_{i,j} \in \{0, 1\}$ 
5: end for
6: return  $(0, b) - \sum_{i=0}^{N-1} \sum_{j=0}^t a_{i,j} \cdot \text{ksk}_{j,i}$ 

```

Algorithm 2 Conversion: from one RLWE ciphertext to one RLWE

Input: $c := \text{FHE.RLWE.Enc}(s, \Delta \cdot M(X)) \in \mathcal{R}_q^2$ and a key switching key $\text{ksk} := \{\text{FHE.ksk.Gen}(s, s_j)\}_{j \in [0, \dots, N-1]}$, where s_j is j -th coefficient of the secret key s , and an index $k \in \{0, \dots, N-1\}$.
Output: $\text{FHE.RLWE.Enc}(s, \Delta \cdot M_k)$

```

1: function KEYSWITCH( $c, \text{ksk}$ )
2:   Run SampleExtract( $c, k$ )  $\rightarrow (a_0, \dots, a_{N-1}, b)$ 
3:   for  $i \leftarrow 0$  to  $N-1$  do
4:      $c_i \leftarrow (\langle g^{-1}(a_i), \text{ksk}_i[1] \rangle, \langle g^{-1}(a_i), \text{ksk}_i[2] \rangle)$ 
5:   end for
6:   return  $c \leftarrow (-\sum_{i=0}^{N-1} c_i[1], b - \sum_{i=0}^{N-1} c_i[2])$ 
7: end function

```

Homomorphic Trace Function The algorithm appeared in [11] for the first time, and it was generalized later in Algorithm 1 of [12] defined as homomorphic trace function. It takes an RLWE ciphertext encrypting a polynomial $M(X)$, and outputs an RLWE ciphertext encrypting the constant term of $M(X)$. For other coefficients, one simply rearranges coefficients of the entire input ciphertext to place the target coefficient to the constant term.

The basic idea is to make half of the coefficients of the underlying message polynomial zero, iteratively, except the constant term, by employing the property of the underlying ring structure, $X^N = -1$. For example, $M(X^{N+1}) = M_0 + M_1 X^{N+1} + M_2 X^{2(N+1)} + M_3 X^{3(N+1)} + \dots + M_{N-1} X^{(N+1)(N-1)}$. Then we can observe that all the even terms remain the same and the sign of the odd terms is changed. Adding $M(X)$ and $M(X^{N+1})$ cancels out all the odd terms due to the opposite sign, therefore, it outputs a polynomial where coefficients of the even terms are doubled.

We repeat the process iteratively by applying $M(X^{N/i+1})$ for the i -th iteration where $i \in [\log N]$, then the output is $N \cdot M_0$. To guarantee the correctness, the message of input is scaled with $1/N$, then the output factor N will be canceled out. In other words, the input ciphertext is an RLWE ciphertext encrypting $M(X)/N$. The algorithm has $\log N$ computational complexity, which is the lowest among others, however the added noise is the biggest, due to the factor N .

Algorithm 3 is improved by [9] for odd prime plaintext modulus p in order to reduce key switching key size. However, constraining p is more about the optimization of the underlying FHE scheme, does not affect the PIR design therefore, the algorithm is not considered in this paper.

Here we summarize the three different conversion algorithms in the following table (see Table 1) to see the pros and cons depending on computation time, memory consumption and the noise growth. And Table 2 shows exact computation cost of each homomorphic operation for readers to be able to estimate how fast each algorithm will be, and compare the speed of the three algorithms. For the noise growth of each algorithm, we

Algorithm 3 Homomorphic Trace: from one RLWE ciphertext to one RLWE

Input: $c := \text{FHE}.\text{RLWE}.\text{Enc}(s(X^k), \Delta \cdot M(X^k)) \in \mathcal{R}_q^2$ and a key switching key $\text{ksk} := \text{FHE}.\text{ksk}.\text{Gen}(s, s(X^k))$, where k is an odd integer.

Output: $\text{FHE}.\text{RLWE}.\text{Enc}(s, \Delta \cdot M(X^k))$

```

1: function KEYSWITCH( $c, \text{ksk}$ )
2:   for  $i \leftarrow 0$  to  $\log N - 1$  do
3:      $c' \leftarrow c(X^{(N/2^i)+1})$ 
4:     Parse  $c' = (c'_0, c'_1)$ 
5:     Compute  $c' := (0, c'_1) - (\mathbf{g}^{-1}(c'_0) \cdot \text{ksk}[1], \mathbf{g}^{-1}(c'_0) \cdot \text{ksk}[2])$ 
6:      $c \leftarrow c + c'$ 
7:   end for
8:   return  $c$ 
9: end function

```

refer to original papers [14,18,11] which has a proof. We note that the dominant operation in Algorithm 3 is FFT conversion from standard domain to FFT domain for polynomial multiplications, since $\mathbf{g}^{-1}(\cdot)$ operation requires standard domain as an input then converting to FFT format is performed to compute dot products efficiently.

Each algorithm has different parameters t and ℓ which are flexibly chosen depending on other parameters n, N, q and multiplicative depth of the protocol, since those two factors are highly related to the noise growth and computation time. If the protocol has low multiplicative depth, therefore the noise caused during the main computation is low enough, then one can set ℓ or t as small as possible to accelerate the corresponding query unpacking algorithm.

	Algorithm 1	Algorithm 2	Algorithm 3
Computation	$N \cdot t$ Hom.Add	$N \cdot \ell$ Scalar.Mult	$\log N \cdot \ell$ FFT
Memory	$O(N \cdot t)$	$O(N \cdot \ell)$	$O(\log N \cdot \ell)$
Noise	$O(N \cdot t)$	$O(N \cdot \ell \cdot g^2)$	$O(N^2 \cdot \ell \cdot g^2)$
Noise budget	31	23	16

Table 1: Comparison of the three RLWE to RLWE conversion algorithms in terms of computation, memory consumption and the noise growth. Hom.Add stands for homomorphic addition, Scalar.Mult stands for scalar multiplication, and FFT stands for FFT conversion. The noise is expressed in the dominant factor in the full variance of the output. The noise budget which indicates how much noise has left before decryption fails, is computed with parameter $q = 2^{64}$, $N = 2048$, $\ell = 2$, $g = 2^{15}$, $t = 61$, and the noise contained in the key switching key has variance 2^9 , yielding 119 bits of security [4] as the original papers of the algorithms used.

	Hom.Add	Scalar.Mult	FFT
Computation time	19 ns	34.69 ns	1954.87 ns

Table 2: Computation time (in nanoseconds) of each homomorphic operation with a machine equipped with dual Intel Xeon Gold 6226 CPUs for $N = 2048$, $q = 2^{64}$.

Conversion from RLWE to RGSW In the above subsection, we studied three different conversion algorithms from an RLWE to an RLWE ciphertext. In order to generate an RGSW ciphertext, we first recall

the structure of RGSW ciphertext. An RGSW ciphertext, defined in Section 2, consists of ℓ RLWE samples where a message is encoded on the first components of them and the other ℓ RLWE samples where the same message is encoded on the second components of them.

In fact, the first ℓ samples are formed as $c_i := (a_i + g^i \cdot m, a_i \cdot s + e_i)$. It can be represented as $(a_i - g^i \cdot m + g^i \cdot m, (a_i - g^i \cdot m) \cdot s + e_i) := (a'_i + g^i \cdot m, a'_i \cdot s + e_i)$. This expression can be easily obtained by an external product between an RLWE ciphertext encrypting m scaled with g^i and RGSW ciphertext encrypting $-s$. In other words, once we have lower ℓ RLWE samples of an RGSW ciphertext, we can easily generate the upper ℓ rows by running ℓ external products one by one, with conversion key which is $\text{FHE} \cdot \text{RGSW} \cdot \text{Enc}(s, -s)$. This technique was introduced in Algorithm 4 of [11].

Algorithm 4 RLWEtoRGSW1: from ℓ RLWE ciphertext to one RGSW

Input: $\{c_i := \text{FHE} \cdot \text{RLWE} \cdot \text{Enc}(s, g^i \cdot m) \in \mathcal{R}_q^2\}_{i \in [\ell]}$ and a conversion key $\text{convk} := \text{FHE} \cdot \text{RGSW} \cdot \text{Enc}(s, -s)$.

Output: $\text{FHE} \cdot \text{RGSW} \cdot \text{Enc}(s, m)$

```

1: for  $i \leftarrow 1$  to  $\ell$  do
2:    $\mathbf{C}[i] \leftarrow \text{ExternalProd}(c_i, \text{convk})$ 
3:    $\mathbf{C}[i + \ell] \leftarrow c_i$ 
4: end for
5: return  $\mathbf{C}$ 

```

Similarly, new representation of the upper ℓ rows of an RGSW ciphertext is introduced recently in [26] (see Algorithm 5 in Appendix A) with smaller conversion key size, resulting in faster running conversion time as well. In other words, their representation is as follows: $c_i := (a' + a_i \cdot s + e_i + g^i \cdot m, (a' + a_i \cdot s + e_i) \cdot s + \tilde{e}) = (\hat{a} + g^i \cdot m, \hat{a} \cdot s + \tilde{e})$, where $\hat{a} = a_i \cdot s + e_i$ looks computationally random due to RLWE assumption, therefore, the new representation of c_i is a valid RLWE sample where the message part $g^i \cdot m$ is encoded on the first component.

In order to achieve that form, additional conversion key is required, encrypting square of the secret key s^2 under s by computing $\text{FHE} \cdot \text{ksk} \cdot \text{Gen}(s, s^2)$. The benefit of this method, again, is that the size of conversion key is reduced by half compared to the method of [11], therefore, the running time of this algorithm is also reduced by half.

We note that both algorithms produces the same level of noise (we refer to the original papers for the proof); $\text{Var}(\text{Err}(\mathbf{C})) = O(N \cdot \ell \cdot \text{Var}(\text{Err}(\mathbf{c})))$, where $\mathbf{c}$ is the input of the conversion (from RLWE to RGSW) algorithms and $\mathbf{C}$ is the output of the algorithm. We also can see that using Algorithm 3 for query unpacking incurs a factor of N^3 in the noise.

5.2 Query unpacking algorithm

We can finally build query unpacking algorithm by combining the above methods, below. We assume that client sends ℓ packed RLWE ciphertexts with different scaling factors g^i for $i \in [\ell]$ to the server.

1. Server parses the query $q := c_1, \dots, c_\ell$.
2. Run RLWE to RLWE conversion algorithm (Algorithm 1, Algorithm 2, or Algorithm 3) for each c_i to obtain $\{c'_{i,j}\}_{i \in [\ell]}$ for $j \in [k]$, where k is the maximum degree of the packed message polynomial.
3. Run RLWE to RGSW conversion algorithm (Algorithm 4 or Algorithm 5) for each $j \in [k]$.
4. Return the output.

Therefore, there are 6 combinations to achieve efficient query unpacking for RLWE based PIR, each of which has different trade-offs. In order to further reduce the communication cost, one can pack ℓ ciphertexts in one ciphertext in the same way, which results in more computation time for query unpacking.

6 A note on parameters

The most important parameters which do not influence security but efficiency are g and ℓ where $g^\ell \leq q$. As we have shown in previous sections, g and ℓ affect the size of ciphertext and the noise growth.

Since we aim to fix the concrete bit security as close to 128, the other parameters such as q, N , and the noise distribution parameters cannot be modified. That is why g and ℓ are considered free parameters, and developers should choose them carefully to achieve the best performance given the fixed security level and the size of the database.

If one increases g , then ℓ can be lowered as long as $g^\ell \leq q$, resulting in smaller ciphertexts (key switching keys, RGSW ciphertexts in general), less computation time. However, it also increases the noise after operations with such ciphertexts. Therefore, both can be chosen differently depending on the multiplicative depth of the target algorithm.

For example, Algorithm 2 and Algorithm 3 output different noise level as shown in Table 1, specifically Algorithm 3 causes more noise than Algorithm 2. Therefore, there is a high chance that Algorithm 2 can use bigger g (lower ℓ) than the other one.

The most important thing to remember in FHE based PIR is that the noise after query unpacking should not be too high so that it can handle the main computation which has $\log n$ multiplicative depth, afterwards. Therefore, we emphasize that analyzing the noise of a protocol is highly important to guarantee correctness with high probability, hence so is providing concrete parameters (g, ℓ) .

Unfortunately, existing works often mention their choice of parameter sets without concrete justification of their choice. In other words, they typically do not demonstrate the remaining noise after computation with the selected parameters. As a result, the selection of the optimal pair of (g, ℓ) and the rationale behind their choice remain unclear.

In Section 8, we will demonstrate how the best parameter set, particularly (g, ℓ) , is selected, and provide concrete values for these parameters, while fixing the other parameters, based on a specific database size.

7 Comparison between the two structures

	Tree-based PIR(SHECS-PIR)	Hypercube-based PIR (dim=2)	Hypercube-based PIR(OnionPIR)
Query unpacking	$\log_2 n \cdot \ell_s$	$2 \cdot \log_2 n \cdot \ell_2$	$(128 + 4 \log_4 (n/128)) \cdot \ell_o$
Main computation	$4 \cdot (n - 1) \cdot \ell_s$	$8 \cdot (n - 1) \cdot \ell_2$	$4 \cdot n \cdot \ell_o + 16/3 \cdot \ell_o(n/128 - 1)$
Main(optimized)	$n + (n/2 - 1) \cdot 4 \cdot \ell_s$	$2 \cdot n + (n - 2) \cdot 4 \cdot \ell_2$	$2 \cdot n + 16/3 \cdot \ell_o(n/128 - 1)$

Table 3: Complexity comparison of computational costs. The gadget vector parameter (ℓ) is denoted separately as ℓ_s, ℓ_2 , and ℓ_o for the corresponding cases. All three of them can be the same or different depending on what algorithm each protocol chooses to implement. We note that the complexity of query unpacking equals the number of query ciphertexts that each protocol requires. Main computation indicates the number of polynomial multiplications required by each protocol. If both protocols use plaintext-ciphertext multiplication in the first operation round of their protocol, the number of ring multiplication can be further reduced as stated in Main(optimized).

In this section, we analyze the differences that arise between the two structures. We assume a communication complexity of $\log n$ for both and compare them in terms of communication cost—measured by the number of ciphertexts required to represent a single query—and main computation cost—measured by the number of polynomial ring operations. A summary of the results is presented in Table 3.

To facilitate this comparison, we generalize the computational cost associated with each structure. The SHECS-PIR structure employs a homomorphic Mux gate to select one element from two. We observed that this Mux gate can be generalized to select one element from $m \geq 2$ candidates. While this generalization increases the communication cost to a level comparable to that of the hypercube-based structure, it leads to a reduction in multiplicative depth.

The hypercube structure, on the other hand, allows each dimension to be configured with a size of 2, yielding an optimal communication cost; only twice that of the tree-based structure as shown in the second column of Table 3.

At first glance, the two structures may appear to behave similarly. However, in the following paragraphs, we highlight the key differences that distinguish them.

7.1 Various optimizations from structural difference

Tree-based PIR We extend the concept of the mux gate to support a more general setting with m encrypted inputs. To be specific, we consider a mux operation that outputs the encryption of the message of the j -th input among m inputs, given encrypted information associated with the index j . We denote this operation by Mux_m . For example, when $m = 4$, the j can be represented using two bits b_0 and b_1 where $j = b_0 + 2b_1$. Let $\mathbf{C}_0$ and $\mathbf{C}_1$ be encryptions of b_0 and b_1 , respectively, and $\mathbf{C}_{01}$ be an encryption of the product b_0b_1 . Then, the Mux_4 operation should output an encryption of $\mathbf{a}_j$, where each $\mathbf{a}_i$ is an encryption of m_i .

$$\begin{aligned} \text{Mux}_4(\mathbf{C}_0, \mathbf{C}_1, \mathbf{C}_{01}, \mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3) \rightarrow \\ \mathbf{C}_{01}(\mathbf{a}_3 - \mathbf{a}_2 + \mathbf{a}_1 - \mathbf{a}_0) + \mathbf{C}_1(\mathbf{a}_2 - \mathbf{a}_0) + \mathbf{C}_0(\mathbf{a}_1 - \mathbf{a}_0) + \mathbf{a}_0 \end{aligned}$$

The above Mux_4 operation is constructed by applying the Mux_2 (i.e., FHE.Mux) twice in a recursive manner. This formula recursively applies conditional selection logic using homomorphic encryption, resulting in an encrypted output of the selected input $\mathbf{a}_j$ based on the encrypted index bits.

Generalizing the idea, Mux_{m^2} operation can be constructed recursively by composing m Mux_m operations. To support this recursive construction efficiently, we assume that m is double-exponential in 2, which ensures that all intermediate layers can be composed without requiring dummy ciphertexts. Here, we use the term double-exponential to refer to functions of the form 2^{2^t} , where t is a non-negative integer.

Each Mux_m operation requires $m - 1$ ciphertexts related to an index. To retrieve the encryption of one input among n data items $(n - 1)/(m - 1)$ invocations of Mux_m are required. The multiplicative depth of this is $\log_m n$.

Hence, a total of $(m - 1) \log_m n$ ciphertexts are required for an index and $(n - 1)$ external products are needed regardless of the value of $m = 2^{2^t}$. Consequently, using FHE.Mux (i.e. Mux_2) is the most efficient choice in tree-based structure for both communication and computation cost.

Hypercube-based PIR In the hypercube structure, multiplying the encryption of a D_i -dimensional unit vector can be viewed as functionally equivalent to a Mux_{D_i} operation, as it corresponds to selecting out of D_i elements. However, the differences in structure result in fundamentally different optimization paths. The hypercube structure offers greater flexibility in choosing the dimensional parameters since it does not need to be double exponential in 2.

Instead, it only requires that the product of the sizes of all dimensions equals the number of database entries. This allows the dimension sizes to be easily modified as needed. We can generalize the way of *OnionPIR* by separating the first dimension (denoted as D_1) and the rest (denoted by m) as follows. The total number of ring multiplications can be expressed as: $4 \cdot n \cdot \ell_o + 4 \cdot \ell_o \cdot \frac{n}{D_1} (1 + 1/m + 1/m^2 + \dots + 1/m^{d-2})$. Suppose the hypercube structure has equal-sized dimensions m , (i.e., $D_1 = D_2 = \dots = D_d = m$, and $d = \log_m n$). Then, the number of polynomial multiplications simplifies to: $4 \cdot \ell_o \cdot m \cdot (n - 1)/(m - 1)$ and the number of ciphertexts required for query unpacking becomes: $m \cdot \log_m n \cdot \ell_o$.

When $m = 2$, as shown in Table 3, their main computation still takes longer than tree-based PIR, however, the communication cost is optimized (still 2x more than tree-based PIR).

However, as we said above, if the dimension size is chosen carefully, the main computation time can be more improved. A good example of such improved parameter selection is demonstrated by *OnionPIR*. We see the difference between the two last columns of Table 3. *OnionPIR*'s parameter choice improves computation time over the base case (when all dimension sizes are 2), as a trade-off, however, the query size is also increased and query unpacking complexity is increased accordingly.

7.2 Computation depending on database type

Both structures can optimize their computation when the database is public. The well-known method is to employ plaintext-ciphertext computation for the first round of CMUX operation or the first dimension as discussed in OnionPIR.

We briefly discuss the case of SHECS-PIR. Instead of using RGSW ciphertext for all $\log n$ bits, the first bit can be sent as an RLWE ciphertext and CMUX for the corresponding round can be done via $\text{RLWE}(\epsilon_0) \cdot (d_i - d_{i-1}) + d_{i-1}$ since the data elements d_i 's are plaintexts. Therefore, we do not need to use expensive ciphertext-ciphertext operations here. In this case, the query unpacking complexity is reduced further from $\log n \cdot \ell_s$ to $(\log n - 1) \cdot \ell_s$ accordingly.

However, this approach incurs additional costs in terms of noise. Specifically, it introduces noise proportional to the size of the data element. As a result, the parameter ℓ may need to be chosen significantly larger than in the base case to ensure correctness. Using the parameters for $n = 2^{20}$ outlined in Section 8, we performed simulations to compare the noise growth between two approaches: one involving ciphertext-ciphertext multiplications for all $\log n$ multiplicative depths, and another where the first multiplicative depth utilized plaintext-ciphertext multiplication followed by ciphertext-ciphertext multiplications for the remaining $\log n - 1$ depths.

Our results indicated that the latter approach produced approximately 1.7 times greater noise than the former. Consequently, the expected improvement in computation time may not be fully realized.

In this case, hypercube based PIR performs better than tree-based PIR in terms of computation due to its flexible choice of parameters (dimension).

However, as private database access is increasingly being considered in the context of cloud-based computation, which has gained significant attention in recent years, we should also consider private database case. For instance, a common scenario involves clients first downloading their encrypted data from the cloud and then sending it to a computing party for machine learning inference or other applications.

In this case, the above optimization method cannot be used. In other words, only ciphertext-ciphertext multiplication is allowed. Under this setting, the tree-based structure outperforms in both communication and computation due to the lowest cost.

8 Performance Evaluation

In this section, we compare the experimental results obtained by applying different unpacking algorithms to SHECS-PIR. Specifically, we analyze two cases: one where Algorithm 2 and Algorithm 4 are used together as unpacking algorithms, and the other where Algorithm 3 and Algorithm 4 are combined. By comparing these two cases, we highlight the relative advantages and disadvantages of Algorithm 2 and Algorithm 3.

One can use Algorithm 5 for further optimization to generate an RGSW ciphertext, however the main computation time is dominated by RLWE to RLWE conversions (Algorithms 1 to 3), therefore, we implemented Algorithm 4 only in our experiment.

Furthermore, based on these experimental results, we evaluate the performance difference compared to Onion-PIR and demonstrate that SHECS-PIR outperforms Onion-PIR. All experiments were conducted on a machine equipped with an Intel Xeon Gold 6226 CPU operating at 2.70 GHz, supporting AVX instructions. Our implementation uses functions defined in [16] and only single core was used. Code is available at <https://github.com/HyewonSung/FHE-based-PIR-revisited.git>.

Noise budget. We demonstrate that how each algorithm affects on the final noise by employing the noise budget [13] which shows how much noise has left until decryption fails. If it is 0, there is no more noise to be consumed; hence, decryption failure will happen highly likely if more operations are done afterwards.

Parameters. We fixed the polynomial degree $N = 2048$, ciphertext modulus $q = 2^{64}$ and the plaintext modulus $p = 2^8$. Thus, our single ciphertext can hold up to 63KB of plaintext data, which is sufficient to encode typical query targets such as URLs and phone numbers. This enables the proposed scheme to be directly applicable to real-world PIR deployments, such as those implemented by Apple [1].

Our parameters (g, ℓ) are selected to achieve optimal performance for each database size, meaning the total computation time is minimized while maintaining a small noise budget close to zero.

8.1 Comparison of Algorithm 2 and Algorithm 3

The original SHECS-PIR used slower unpacking (Algorithm 1). We first modified the original SHECS-PIR protocol by replacing only the query-unpacking phase with two alternative combinations: Algorithm 2 combined with Algorithm 4, and Algorithm 3 combined with Algorithm 4.

In our experiments, we measured the unpacking time, main computation time, and total protocol execution time across various database sizes. Additionally, by adjusting parameters (g, ℓ) , we identified the parameter combinations that minimized the total computation time and the noise budget. Table 4 summarizes the computation costs for the combination of Algorithm 2 and Algorithm 4, while Table 5 presents the results obtained using Algorithm 3 and Algorithm 4.

Computation Time Comparison. Algorithm 2 internally involves scalar multiplication operations, whereas Algorithm 3 internally involves FFT conversions. As shown in Table 2, scalar multiplication is substantially faster than FFT conversion. However, since Algorithm 2 must perform scalar multiplication operations $N \cdot \ell$ times compared to $\log N \cdot \ell$ FFT conversions required by Algorithm 3, the unpacking time of Algorithm 2 becomes considerably longer.

Comparing the results in Table 4 and Table 5, the unpacking time using Algorithm 3 is consistently below 1 second, whereas Algorithm 2 typically exceeds 2 seconds. Although this longer unpacking time might initially appear disadvantageous for Algorithm 2, it becomes negligible as the database size increases (*e.g.*, $n \geq 2^{20}$). In such scenarios, the server’s main computation, dominated by CMUX gate operations, accounts for most of the total execution time, making the unpacking time insignificant.

The difference in main computation time between Algorithm 2 and Algorithm 3 for the same database size arises primarily due to the parameter ℓ . Specifically, Algorithm 3 has rapid error growth and therefore requires choosing a large ℓ to avoid decryption failure. In contrast, Algorithm 2 exhibits slower error growth, enabling the selection of a relatively small value (*e.g.*, $\ell = 3$). Since a higher ℓ increases the size of RGSW ciphertexts and thus the CMUX computation time, Algorithm 3 consequently exhibits substantially longer main computation times despite its fast unpacking. Conversely, the smaller ℓ chosen in Algorithm 2 significantly reduces its main computation time.

As a result, Algorithm 2 and Algorithm 3 demonstrate comparable overall execution times for smaller databases (*e.g.*, $n = 2^{16}$). However, as the database size grows larger ($n = 2^{20}$ or more), Algorithm 2 significantly outperforms Algorithm 3 in terms of total execution time.

Therefore, using Algorithm 3 is not always the best in query unpacking even though its optimal computation complexity. We recommend that the based applications employing this algorithm reevaluate their choice considering their main computation time and the final noise as we did.

Noise Budget Comparison and Analysis. Tables 4 and 5 illustrate a significant difference in the noise budgets between Algorithm 2 and Algorithm 3. Typically, the noise budget of Algorithm 3 approaches 0, whereas Algorithm 2 maintains a relatively larger noise budget. This difference arises from their differing error growth rates.

Due to the rapid error growth in Algorithm 3, the noise budget quickly diminishes to 0 for most parameter combinations (g, ℓ) , substantially increasing the risk of decryption failure. Conversely, the slower error growth of Algorithm 2 ensures the noise budget remains sufficiently large, significantly broadening the range of viable parameters that avoid decryption failures.

Moreover, we observed a gradual decline in the noise budget for Algorithm 2 as the database size increased. Given this trend, for databases larger than 2^{25} , Algorithm 2 will offer considerably more parameter flexibility and safety against decryption failures compared to Algorithm 3.

Optimal Parameters for Each Algorithm. The primary objective of our experiments was to identify optimal parameters for each algorithm based on two criteria:

1. minimizing the total computation time, and
2. minimizing the noise budget.

Each algorithm’s strengths and weaknesses are summarized as follows:

- **Algorithm 2** has a relatively slow unpacking time but benefits from slow error growth, which allows the use of a smaller ℓ . A smaller ℓ leads to reduced main computation time.
- **Algorithm 3** offers extremely fast unpacking but suffers from rapid error growth, necessitating a larger ℓ , which increases the main computation time.

Due to these differences, the approach to determining optimal parameters varies for each algorithm. For Algorithm 3, because the noise budget typically reaches zero, optimization relies solely on minimizing total execution time. In contrast, Algorithm 2 preserves a larger noise budget, enabling simultaneous optimization of both total execution time and the noise budget. Consequently, Algorithm 2 allows for more comprehensive parameter optimization compared to Algorithm 3.

By adopting this optimization approach for Algorithm 2, we successfully minimized ℓ to as low as 3, significantly reducing both computation time and noise budget. However, achieving such a small ℓ required selecting a large g , typically $g \geq 2^{16}$. Among various tested values of g , we recorded in Table 4 the parameters yielding the fastest computation times for each database size.

On the other hand, although Algorithm 3 inherently required higher values of ℓ , we managed to minimize it effectively to around 6 or 7, with corresponding g values of approximately 2^6 or 2^7 . The parameters for Algorithm 3 achieving the fastest computation times across different database sizes are summarized in Table 5.

	Algorithm 2 + 4			
Database size n	2^{16}	2^{20}	2^{23}	2^{25}
(g, ℓ)	$(2^{18}, 3)$	$(2^{19}, 3)$	$(2^{17}, 3)$	$(2^{16}, 3)$
unpacking time (sec)	2.0	2.4	2.9	6.8
main computation (sec)	2.1	33.4	346.1	3600
total computation (sec)	4.2	36.1	349.2	3607
Noise budget	8	9	7	2

Table 4: This table summarizes the fastest results obtained when applying Algorithm 2 and Algorithm 4 as unpacking algorithms in SHECS-PIR across various database sizes. Here, the main computation time refers to the time required for the server to perform $n - 1$ CMUX gate operations, and the total execution time represents the overall time needed to run the protocol once.

	Algorithm 3 + 4			
Database size n	2^{16}	2^{20}	2^{23}	2^{25}
(g, ℓ)	$(2^7, 7)$	$(2^7, 6)$	$(2^7, 6)$	$(2^6, 7)$
unpacking time (sec)	0.1	0.1	0.1	0.4
main computation (sec)	3.7	55.0	523.6	4380
total computation (sec)	4.6	55.7	524.4	4381.6
Noise budget	0	0	0	1

Table 5: This table summarizes the fastest results obtained when applying Algorithm 3 and Algorithm 4 as unpacking algorithms in SHECS-PIR across various database sizes.

8.2 Comparison with the original SHECS-PIR and OnionPIR

Here, we provide a fair comparison between two implementations of the different structures (SHECS-PIR and OnionPIR). As we mentioned in Section 1, there is a misconception that tree-based structure performs less than hypercube-based PIR since the choice of query unpacking algorithm of SHECS-PIR is outdated than what OnionPIR selected.

In the previous section, we already showed that the misconception is wrong in the case of private database. Even in the case of parameters are carefully chosen (OnionPIR), still tree-based PIR outperforms in both communication and computation. In the following, we show experimental result which uses the same algorithms for each, assuming the same parameters (especially g, ℓ) since the noise growth of both cases is $O(\log n)$.

		SHECS-PIR						OnionPIR ([34])	
unpacking method		Algorithm 1 ([36])		Algorithm 2		Algorithm 3		Algorithm 3	
Parameters	Database size n	2^{16}	2^{20}	2^{16}	2^{20}	2^{16}	2^{20}	2^{16}	2^{20}
	(g, ℓ)	$(2^{15}, 2)$	$(2^{15}, 2)$	$(2^{18}, 3)$	$(2^{19}, 2)$	$(2^7, 7)$	$(2^7, 6)$	$(2^7, 7)$	$(2^7, 6)$
Memory	key-switching key (MB)	262144	262144	192	128	2.4	2.1	2.4	2.1
Communication cost	# of packed-ciphertexts	2	2	3	2	7	6	7	6
	packed-query size (KB)	64	64	96	64	224	192	224	192
	# of ciphertexts	32	40	48	40	112	120	1022	924
	query size (KB)	1024	1280	1536	1280	3584	3840	32,688	29,568
Computation cost	unpacking time (sec)	4.9	5.9	2.0	2.4	0.1	0.1	0.8	0.7
	main computation (sec)	3.0	73.6	2.1	33.4	3.7	55.0	3.7	55.5

Table 6: This table compares the performance of SHECS-PIR with various unpacking methods against that of OnionPIR, which uses Algorithm 3 as its unpacking method. For SHECS-PIR, the parameters listed were selected to achieve optimal performance for each database size. The memory row indicates the size of the key-switching key required by each algorithm. The communication cost denotes the number of ciphertexts the client must send to the server to query a single index and is reported both with and without ciphertext packing. The computation cost is split into unpacking time and the server’s main computation time.

Our experimental result for SHECS-PIR is discussed in Tables 4 and 5. OnionPIR used multi-threading techniques and parameters N, q are larger than our choice, also the choice of (g, ℓ) is not clearly stated, therefore we calculated their computation time by considering each algorithm speed (Table 2) and the number of polynomial multiplications Table 3 that we analyzed.

Table 6 summarizes a comparative analysis of SHECS-PIR with various unpacking algorithms and OnionPIR. This table not only facilitates the comparison of OnionPIR with SHECS-PIR using the same unpacking method but also highlights the performance differences when applying diverse unpacking algorithms to tree-based PIR. Below, we analyze each criterion presented in the table in detail.

Memory Consumption. Memory consumption refers to the storage required for the key-switching keys of each algorithm. As indicated in Table 1, Algorithm 1 requires $N \cdot t$ keys consisting of RLWE ciphertexts, Algorithm 2 requires N keys in the form of RGSW ciphertexts, and Algorithm 3 requires $\log n$ keys also in the form of RGSW ciphertexts. Since the ciphertext modulus q is fixed at 2^{64} , an RLWE ciphertext—which contains two polynomials of degree N —requires $16N$ bytes. Similarly, an RGSW ciphertext consisting of 2ℓ RLWE ciphertexts requires $32\ell N$ bytes. Using these calculations, the key-switching keys for Algorithm 1 occupy the largest memory, whereas Algorithm 3 has the smallest requirement, approximately 2MB. These key-switching keys must be pre-generated in the offline phase before executing the protocol, and the total offline memory consumption grows significantly as the number of required keys increases.

Communication Cost. Communication cost in this context refers to the number of ciphertexts a client should transmit to the server when querying a single database index after protocol initialization. A higher number of ciphertext transmissions implies greater communication overhead. We measured this communication cost separately for two scenarios: with and without ciphertext packing optimization. It is well known that both SHECS-PIR and Onion-PIR have the same asymptotically optimal communication complexity of $\mathcal{O}(\log n)$. Therefore, when ciphertext packing is employed, both protocols are supposed to exhibit identical optimal communication costs. However, without using ciphertext packing, OnionPIR incurs additional ciphertext transmissions due to the requirement of sending extra ciphertexts per dimension of the hypercube, as explained in Section 7. In contrast, SHECS-PIR consistently maintains an optimal communication cost, requiring only $\log n$ ciphertext transmissions even without packing optimization. In practice (employing current state-of-the-art FHE), due to the use of gadget decomposition, the actual number of ciphertexts sent to the server is multiplied by the parameter ℓ , which is why the number of packed ciphertexts are proportional to the value of ℓ and so is the actual size in the table.

Computation Cost. The computation cost refers the server’s computation workload, which we measured separately as the unpacking time and the main computation time. A detailed comparison between Algorithm 2 and Algorithm 3 within SHECS-PIR is provided in Section 8.1. Compared to the results using Algorithm 1 (the original unpacking algorithm applied in [36]), significant reductions in unpacking time were observed when replacing Algorithm 1 with either Algorithm 2 or Algorithm 3. This highlights that the choice of unpacking algorithm substantially impacts the computation cost of tree-based PIR schemes.

When comparing OnionPIR with SHECS-PIR employing Algorithm 3 as the unpacking method, a slight difference emerges in unpacking times. This discrepancy arises from the difference in the number of times Algorithm 3 is executed between OnionPIR with SHECS-PIR. Specifically, Algorithm 3 is executed once per bit encoding the queried index within the ciphertext. For SHECS-PIR, since the database index is represented precisely using $\log n$ bits, Algorithm 3 is executed exactly $\log n$ times.

On the other hand, OnionPIR, as illustrated in Table 3, encodes the index with $128 + 4 \cdot \log_4(n/128)$ bits, resulting in slightly more executions of Algorithm 3, thereby increasing unpacking time.

The main computation refers to the time the server requires to retrieve the value corresponding to the queried index from the database. In total, the main computation time dominates as the database size increases. However, as we can see from Table 6, when the database size is 2^{16} , still the computational overhead caused by unpacking plays a significant role. The little overhead of tree-based PIR does not increase much as we can see in Tables 4 and 5 as the database size increases regardless of a choice of unpacking algorithm. If the database size is small enough (up to 2^{20}), tree-based PIR would give the best performance in terms of both computation (when packing method is used) and communication (without packing method).

9 Discussion and Conclusion

9.1 Discussion

Noise budget If there is some noise left after the whole protocol, then it means there are more operations possible, and it also means that decryption failure probability is small, so that correctness is guaranteed with high probability. In other words, if the noise budget is close to 0, decryption failure probability becomes higher, therefore, it is not always the best idea to optimize parameters to reach such a small noise budget. Therefore, it is always recommended to have a small room at the end in the noise to guarantee high correctness probability.

9.2 Conclusion

In this paper, we revisited single server PIR based on fully homomorphic encryption, in order to analyze which protocol performs better with what optimization methods. To this end, we first categorized existing protocols into two classes depending on their structure. Secondly, we introduced existing homomorphic algorithms which can be used for query optimization and compare them in terms of computation complexity, and noise growth. Lastly, we provided a detailed comparison between the two different categories, and show which structure performs better in which scenario.

References

1. Combining machine learning and homomorphic encryption in the apple ecosystem (2024), <https://machinelearning.apple.com/research/homomorphic-encryption>
2. Aguilar Melchor, C., Barrier, J., Fousse, L., Killijian, M.O.: XPIR : Private Information Retrieval for Everyone. *Proceedings on Privacy Enhancing Technologies* **avril 2016**, 155–174 (Apr 2016). <https://doi.org/10.1515/popets-2016-0010>, <https://hal.science/hal-01396142>
3. Aguilar-Melchor, C., Barrier, J., Fousse, L., Killijian, M.O.: XPIR: Private information retrieval for everyone. *PoPETs* **2016**(2), 155–174 (Apr 2016). <https://doi.org/10.1515/popets-2016-0010>
4. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology* **9**(3), 169–203 (2015). <https://doi.org/doi:10.1515/jmc-2015-0016>, <https://doi.org/10.1515/jmc-2015-0016>
5. Angel, S., Chen, H., Laine, K., Setty, S.T.V.: PIR with compressed queries and amortized query processing. In: 2018 IEEE Symposium on Security and Privacy. pp. 962–979. IEEE Computer Society Press (May 2018). <https://doi.org/10.1109/SP.2018.00062>
6. Bonte, C., Iliashenko, I., Park, J., Pereira, H.V.L., Smart, N.P.: FINAL: Faster FHE instantiated with NTRU and LWE. In: Agrawal, S., Lin, D. (eds.) *ASIACRYPT 2022, Part II*. LNCS, vol. 13792, pp. 188–215. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22966-4_7
7. Borisov, N., Danezis, G., Goldberg, I.: DP5: A private presence service. *PoPETs* **2015**(2), 4–24 (Apr 2015). <https://doi.org/10.1515/popets-2015-0008>
8. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. In: Goldwasser, S. (ed.) *ITCS 2012*. pp. 309–325. ACM (Jan 2012). <https://doi.org/10.1145/2090236.2090262>
9. de Castro, L., Lewi, K., Suh, E.: WhisPIR: Stateless private information retrieval with low communication. *Cryptology ePrint Archive*, Paper 2024/266 (2024), <https://eprint.iacr.org/2024/266>
10. Celi, S., Davidson, A.: Call me by my name: Simple, practical private information retrieval for keyword queries. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. p. 4107–4121. CCS ’24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3658644.3670271>, <https://doi.org/10.1145/3658644.3670271>
11. Chen, H., Chillotti, I., Ren, L.: Onion ring ORAM: Efficient constant bandwidth oblivious RAM from (leveled) TFHE. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) *ACM CCS 2019*. pp. 345–360. ACM Press (Nov 2019). <https://doi.org/10.1145/3319535.3354226>
12. Chen, H., Dai, W., Kim, M., Song, Y.: Efficient homomorphic conversion between (ring) LWE ciphertexts. In: Sako, K., Tappenhauer, N.O. (eds.) *ACNS 21 International Conference on Applied Cryptography and Network Security, Part I*. LNCS, vol. 12726, pp. 460–479. Springer, Cham (Jun 2021). https://doi.org/10.1007/978-3-030-78372-3_18
13. Chen, H., Laine, K., Player, R.: Simple encrypted arithmetic library - SEAL v2.2 (2017), technical report
14. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster packed homomorphic operations and efficient circuit bootstrapping for TFHE. In: Takagi, T., Peyrin, T. (eds.) *ASIACRYPT 2017, Part I*. LNCS, vol. 10624, pp. 377–408. Springer, Cham (Dec 2017). https://doi.org/10.1007/978-3-319-70694-8_14
15. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology* **33**(1), 34–91 (Jan 2020). <https://doi.org/10.1007/s00145-019-09319-x>
16. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption library (August 2016), <https://github.com/tfhe/experimental-tfhe/tree/master/circuit-bootstrapping>
17. Chor, B., Goldreich, O., Kushilevitz, E., Sudan, M.: Private information retrieval. In: 36th FOCS. pp. 41–50. IEEE Computer Society Press (Oct 1995). <https://doi.org/10.1109/SFCS.1995.492461>
18. Cong, K., Das, D., Nicolas, G., Park, J.: Panacea: Non-interactive and stateless oblivious ram. *Cryptology ePrint Archive*, Paper 2023/274 (2023), <https://eprint.iacr.org/2023/274>, <https://eprint.iacr.org/2023/274>
19. Cong, K., Geelen, R., Kang, J., Park, J.: Revisiting oblivious top-\$\$\$selection with applications to secure \$\$\$-nn classification. In: Eichlseder, M., Gambs, S. (eds.) *Selected Areas in Cryptography – SAC 2024*. pp. 3–25. Springer Nature Switzerland, Cham (2025)
20. Dams, D., Lataille, J., Sanchez, R., Wade, J.: WIDSEAS: A lattice-based PIR scheme implemented in EncryptedQuery. *Cryptology ePrint Archive*, Report 2019/855 (2019), <https://eprint.iacr.org/2019/855>
21. Davidson, A., Pestana, G., Celi, S.: FrodoPIR: Simple, scalable, single-server private information retrieval. *PoPETs* **2023**(1), 365–383 (Jan 2023). <https://doi.org/10.56553/popets-2023-0022>
22. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, Paper 2012/144 (2012), <https://eprint.iacr.org/2012/144>, <https://eprint.iacr.org/2012/144>
23. Gentry, C., Halevi, S.: Compressible FHE with applications to PIR. In: Hofheinz, D., Rosen, A. (eds.) *TCC 2019, Part II*. LNCS, vol. 11892, pp. 438–464. Springer, Cham (Dec 2019). https://doi.org/10.1007/978-3-030-36033-7_17

24. Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013, Part I. LNCS, vol. 8042, pp. 75–92. Springer, Berlin, Heidelberg (Aug 2013). https://doi.org/10.1007/978-3-642-40041-4_5
25. Henzinger, A., Hong, M.M., Corrigan-Gibbs, H., Meiklejohn, S., Vaikuntanathan, V.: One server for the price of two: Simple and fast single-server private information retrieval. In: Proceedings of the 32nd USENIX Conference on Security Symposium. SEC '23, USENIX Association, USA (2023)
26. Kim, A., Lee, Y., Deryabin, M., Eom, J., Choi, R.: Lfhe: Fully homomorphic encryption with bootstrapping key size less than a megabyte. Cryptology ePrint Archive, Paper 2023/767 (2023), <https://eprint.iacr.org/2023/767>, <https://eprint.iacr.org/2023/767>
27. Kogan, D., Corrigan-Gibbs, H.: Private blacklist lookups with checklist. In: Bailey, M., Greenstadt, R. (eds.) USENIX Security 2021. pp. 875–892. USENIX Association (Aug 2021)
28. Kwon, A., Lazar, D., Devadas, S., Ford, B.: Riffle: An efficient communication system with strong anonymity. PoPETs **2016**(2), 115–134 (Apr 2016). <https://doi.org/10.1515/popets-2016-0008>
29. Li, B., Micciancio, D., Raykova, M., Schultz, M.: Hintless single-server private information retrieval. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part IX. LNCS, vol. 14928, pp. 183–217. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68400-5_6
30. Lipmaa, H., Pavlyk, K.: A simpler rate-optimal CPIR protocol. In: Kiayias, A. (ed.) FC 2017. LNCS, vol. 10322, pp. 621–638. Springer, Cham (Apr 2017). https://doi.org/10.1007/978-3-319-70972-7_35
31. Luo, M., Liu, F.H., Wang, H.: Faster FHE-based single-server private information retrieval. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024. pp. 1405–1419. ACM Press (Oct 2024). <https://doi.org/10.1145/3658644.3690233>
32. Menon, S.J., Wu, D.J.: SPIRAL: Fast, high-rate single-server PIR via FHE composition. In: 2022 IEEE Symposium on Security and Privacy. pp. 930–947. IEEE Computer Society Press (May 2022). <https://doi.org/10.1109/SP46214.2022.9833700>
33. Mittal, P., Olumofin, F.G., Troncoso, C., Borisov, N., Goldberg, I.: PIR-tor: Scalable anonymous communication using private information retrieval. In: USENIX Security 2011. USENIX Association (Aug 2011)
34. Mughees, M.H., Chen, H., Ren, L.: OnionPIR: Response efficient single-server PIR. In: Vigna, G., Shi, E. (eds.) ACM CCS 2021. pp. 2292–2306. ACM Press (Nov 2021). <https://doi.org/10.1145/3460120.3485381>
35. Naehrig, M., Lauter, K., Vaikuntanathan, V.: Can homomorphic encryption be practical? In: Proceedings of the 3rd ACM Workshop on Cloud Computing Security Workshop. p. 113–124. CCSW '11, Association for Computing Machinery, New York, NY, USA (2011). <https://doi.org/10.1145/2046660.2046682>, <https://doi.org/10.1145/2046660.2046682>
36. Park, J., Tibouchi, M.: SHECS-PIR: Somewhat homomorphic encryption-based compact and scalable private information retrieval. In: Chen, L., Li, N., Liang, K., Schneider, S.A. (eds.) ESORICS 2020, Part II. LNCS, vol. 12309, pp. 86–106. Springer, Cham (Sep 2020). https://doi.org/10.1007/978-3-030-59013-0_5
37. Patel, S., Persiano, G., Yeo, K.: Private stateful information retrieval. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) ACM CCS 2018. pp. 1002–1019. ACM Press (Oct 2018). <https://doi.org/10.1145/3243734.3243821>
38. Stern, J.P.: A new efficient all-or-nothing disclosure of secrets protocol. In: ASIACRYPT'98. pp. 357–371. Springer (1998). https://doi.org/10.1007/3-540-49649-1_28
39. Xia, H., Wang, M.: LightPIR: Single-server PIR via FHE without gaussian noise. In: Zhou, J., Quek, T.Q.S., Gao, D., Cárdenas, A.A. (eds.) ASIACCS 24. ACM Press (Jul 2024). <https://doi.org/10.1145/3634737.3661140>

A Another algorithm for RLWE to RGSW

The following algorithm is used for RLWE to RGSW conversion. As discussed in Section 5, it can replace Algorithm 4, resulting in better computation cost.

Algorithm 5 RLWEtoRGSW2: from ℓ RLWE ciphertext to one RGSW

Input: $\{c_i := \text{FHE}.\text{RLWE}.\text{Enc}(s, g^i \cdot m) \in \mathcal{R}_q^2\}_{i \in [\ell]}$ and a conversion key $\text{convk} := \text{FHE}.\text{ksk}.\text{Gen}(s, s^2)$.
Output: $\text{FHE}.\text{RGSW}.\text{Enc}(s, m)$

```
1: for  $i \leftarrow 1$  to  $\ell$  do  
2:    $\mathbf{C}[i] \leftarrow (0, c_i[2]) - (\mathbf{g}^{-1}(c_i[1]) \cdot \text{ksk}[1], \mathbf{g}^{-1}(c_i[1]) \cdot \text{ksk}[2])$   
3:    $\mathbf{C}[i + \ell] \leftarrow c_i$   
4: end for  
5: return  $\mathbf{C}$ 
```
